

代码导读

从初始相位到 SLM 输出的核心算法逐段解释

RTAD MRAF/GS Pipeline

2026 年 5 月 30 日

摘要

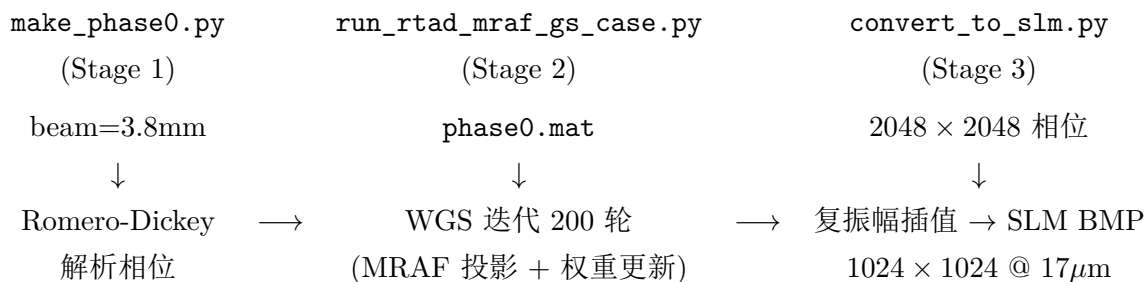
读完这份文档，你就能理解从初始相位到 SLM 输出的每一行关键代码在做什么。所有代码片段均从本目录的 .py 文件中直接提取。

目录

1	管线数据流	2
2	目标生成— $330 \times 120\mu\text{m}$ 矩形平顶	2
2.1	升余弦边缘	2
2.2	四层 Mask 区域	3
3	初始相位—Romero-Dickey 解析法	3
4	DOE 面的输入光场	4
5	傅里叶透镜传播	5
6	MRAF 焦面投影—三个区域，三种规则	5
7	WGS 权重更新—平顶内的局部修匀	6
7.1	直观理解	7
8	主迭代循环—完整流程	7
9	ROM $\rightarrow$ SLM —复振幅插值	8
10	闪耀光栅—让光斑偏离中心	9
11	指标计算—怎么判断做得好不好	9
12	参数速查表	10

1 管线数据流

整个系统分为三个阶段：



三个 Stage 的关系：

- **Stage 1** 生成初始相位猜测（Romero-Dickey 解析法），输出 `phase0.mat`。
- **Stage 2** 用 MRAF + WGS 迭代 200 轮优化相位，记录收敛曲线和指标。
- **Stage 3** 把 2048 × 2048 的优化相位插值缩放到 1024 × 1024，叠加闪耀光栅，输出 SLM 所需的 BMP 文件。

2 目标生成—330 × 120μm 矩形平顶

文件: `src/rtad_target.py`

2.1 升余弦边缘

矩形不是硬边界，而是通过升余弦 (raised cosine) 从 flat 区平滑过渡到零：

```

1 # src/rtad_target.py, line 69-78
2 def raised_cosine_edge(u, u0, u1):
3     """一维升余弦下降沿: u<=u0->1, u>=u1->0, 中间余弦过渡."""
4     C = np.zeros_like(u, dtype=np.float32)
5     C[u <= u0] = 1.0                # flat 区 = 1
6     idx = (u > u0) & (u < u1)
7     t = (u[idx] - u0) / (u1 - u0)   # 归一化到 [0,1]
8     C[idx] = 0.5 * (1.0 + np.cos(np.pi * t)) # 余弦曲线从 1 降到 0
9     return C

```

物理含义：这是一维 profile。对于 330 × 120μm 的目标：

- $a_{50} = 165\mu\text{m}$ (X 方向半宽, 50% 强度处)
- $a_0 = a_{50} - \delta_x = 150\mu\text{m}$ (flat 区边界, 强度 = 1)
- $a_1 = a_{50} + \delta_x = 180\mu\text{m}$ (过渡区外边界, 强度 = 0)
- $\delta_x = 15\mu\text{m}$ 控制边缘过渡的陡峭程度

二维模板 $I(x, y) = C_x(|x|) \times C_y(|y|)$:

```

1 # src/rtad_target.py, line 151-154

```

```

2 Ix = raised_cosine_edge(absX, a0, a1)
3 Iy = raised_cosine_edge(absY, b0, b1)
4 I_full = np.clip(Ix * Iy, 0.0, 1.0)    # 二维强度模板 (可分离)
5 A_full = np.sqrt(I_full)              # 振幅 = sqrt(强度)

```

2.2 四层 Mask 区域

MRAF 算法的核心是把焦面分成四个区，每个区有不同约束规则：

```

1 # src/rtad_target.py, line 161-171
2 # 从内到外四个区域：
3 mask_flat = (absX <= a0) & (absY <= b0)    # 平顶核心 (150x52 um)
4 mask_signal = I_full >= release_level      # 约束区 ( $I \geq e^{-2}$ )
5 mask_signal = mask_signal | mask_flat      # 确保 flat 一定在 signal 内
6 mask_free = mask_guard_window & ~mask_signal # 自由区 (可随意演化)
7 mask_bg_far = ~(mask_signal | mask_free)   # 远背景 (轻微衰减)

```

区域示意图 (以 X 方向为例)：

```

强度
1.0 | xxxxxxxxxxxxxx
    | x flat x\
0.5 | x (a0=150) x\___ 升余弦过渡 ___
    | x          x      \
e^{-2} | xxxxxxxxxxxxxxxxxxxxxxxx\_____
    | |<-- signal -->|          \
0    | |<--          guard_window          -->|
    | |<--    free    -->|<-- bg_far -->|

```

<-- 区域边界: a0=150, a50=165, a1=180, a2=200um -->

3 初始相位—Romero-Dickey 解析法

文件: make_phase0.py

DOE 初始相位不能用随机值——那样迭代根本收不敛。Romero-Dickey 方法从高斯光束和矩形平顶的能量守恒关系出发，解析求出可分离的初始相位：

```

1 # make_phase0.py, line 44-51
2 def romero_dickey_phase_1d(x_m, ri_m, Ro_m, lambda_m, f_m):
3     """Romero-Dickey 一维解析相位."""
4     xi = x_m / ri_m                                # 归一化坐标 (ri=1/e 振幅半径)
5     # 无量纲相位函数:
6     phi_dimless = (xi * np.sqrt(np.pi) / 2.0 * erf(xi)

```

```

7         + 0.5 * np.exp(-xi**2) - 0.5)
8     # beta: 无量纲参数, 表示几何光学近似程度
9     beta = 2.0 * np.pi * ri_m * Ro_m / (lambda_m * f_m)
10    # beta x phi_dimless -> 真实相位 (rad)
11    return beta * phi_dimless

```

β 的物理含义:

$$\beta = 2\pi \times \frac{(\text{光束半径}) \times (\text{目标半宽})}{\lambda \times (\text{焦距})}$$

- $\beta > 10 \rightarrow$ 几何光学区, 稳相近似精确, WGS 只需小幅修正
- $\beta < 10 \rightarrow$ 衍射区, 初始相位偏差大, WGS 需要大幅修正

对于 beam=3.8mm, $f = 200\text{mm}$: $\beta_x = 14.8$, $\beta_y = 5.4$ 。X 方向在几何光学区, Y 方向在衍射区边缘。

二维相位是可分离的: $\phi(x, y) = \phi_x(x) + \phi_y(y)$ 。

4 DOE 面的输入光场

文件: src/propagation.py

```

1  # src/propagation.py, line 48-66
2  def make_input_gaussian(shape, dx_doe_m, gaussian_1e2_diameter_m,
3                          clear_aperture_m, xp, dtype):
4      """构建 DOE 平面高斯输入振幅."""
5      Ny, Nx = shape
6      # 坐标网格 (axis 0 = y, axis 1 = x)
7      x = (xp.arange(Nx, dtype=dtype) - Nx//2) * dtype(dx_doe_m)
8      y = (xp.arange(Ny, dtype=dtype) - Ny//2) * dtype(dx_doe_m)
9      X, Y = xp.meshgrid(x, y)
10     r2 = X*X + Y*Y
11
12     w = dtype(gaussian_1e2_diameter_m / 2.0) # 1/e^2 强度半径
13     amp = xp.exp(-r2 / (w * w))              # 高斯振幅 A=exp(-r^2/w^2)
14
15     # 圆形通光孔径截断 (直径 15mm)
16     aperture_radius = dtype(clear_aperture_m / 2.0)
17     amp = xp.where(r2 <= aperture_radius**2, amp, 0.0)
18
19     # 总功率归一化到 1
20     return normalize_power(amp, xp, target_power=1.0)

```

关键点: $A(r) = \exp(-r^2/w^2)$ ——注意是强度 I 的 $1/e^2$ 半径, 对应振幅的 $1/e$ 半径。 $w = 1/e^2$ 强度直径/2 = 1.9mm (对 beam=3.8mm)。

5 傅里叶透镜传播

文件: src/propagation.py

DOE 面和焦面之间通过一个傅里叶透镜耦合。前向传播 = 从 DOE 到焦面（我们看到的），反传 = 从焦面回到 DOE（迭代中用来更新相位）。

```

1 # src/propagation.py, line 13-16
2 def forward_fft(field_in, xp):
3     """DOE 平面 -> 焦平面: FFT."""
4     # ifftshift 把零频移到 (0,0) -> fft2 -> fftshift 把零频移回中心
5     return xp.fft.fftshift(xp.fft.fft2(xp.fft.ifftshift(field_in), norm="ortho"))
6
7 def backward_fft(field_out, xp):
8     """焦平面 -> DOE 平面: 逆 FFT."""
9     return xp.fft.fftshift(xp.fft.ifft2(xp.fft.ifftshift(field_out), norm="ortho"))

```

关键约定:

- norm="ortho": 保证 FFT 前后总功率守恒
 - 矩阵方向: axis 0 = y (行), axis 1 = x (列)——和 MATLAB meshgrid 一致
 - 焦面坐标: $x_{\text{focal}}[k] = (k - N/2) \times \text{focal_dx}$, 其中 $\text{focal_dx} = 2.5\mu\text{m}$
- 焦面像素间距: $\text{focal_dx} = \lambda \times f / (N \times \text{dx_doe}) = 2.5\mu\text{m}$, 2048×2048 的焦面覆盖 $\pm 2.56\text{mm}$ 。

6 MRAF 焦面投影—三个区域，三种规则

文件: src/mraf_gs.py

这是每次迭代里最关键的一步: 把计算出的焦面复振幅”投影”到物理约束上。

```

1 # src/mraf_gs.py, line 287-326 (精简, 去掉已注释的 GS 分支)
2 def _project_farfield(farfield, target_amp_eff, masks_b, xp, method,
3                       mraf_factor, bg_mode, bg_factor):
4     """MRAF 焦面投影."""
5     # 提取当前焦面相位
6     phase_ff = xp.angle(farfield)
7     phase_factor = xp.exp(1j * phase_ff)
8     signal = masks_b["mask_signal"]
9
10    projected = farfield.copy()
11    # (1) signal 区: 保留相位, 替换振幅为目标振幅
12    projected[signal] = target_amp_eff[signal] * phase_factor[signal]
13
14    # (2) free 区: 保留相位, 振幅 x mraf_factor (0.4)
15    free = masks_b["mask_free"]
16    projected[free] *= mraf_factor

```

```

17
18 # (3) 远背景: keep / attenuate / zero
19 bg = masks_b.get("mask_bg_far", ...)
20 if bg_mode == "attenuate":
21     projected[bg] *= bg_factor          # x 0.9, 几乎等同 keep
22 elif bg_mode == "zero":
23     projected[bg] = 0
24
25 return projected

```

三区处理总结:

区域	振幅处理	相位处理	效果
mask_signal	替换为 $\text{target_amp} \times \text{WGS_weight}$	保留原相位	强制趋近目标平顶
mask_free	$\times 0.4$ (衰减)	保留原相位	给噪声自由度, 不完全压制
mask_bg_far	$\times 0.9$ (几乎不变)	保留原相位	不做强约束, 避免能量倒灌

为什么 `mraf_factor=0.4` 而不是 0: 如果 free 区完全置零, 能量无处可去, 会回流到 signal 区形成波纹。给 free 区一定自由度反而让平顶更平。

7 WGS 权重更新—平顶内的局部修匀

文件: `src/mraf_gs.py`

MRAF 只做区域级约束。WGS 在 `mask_flat` 内部做像素级权重反馈, 进一步压平不均匀。

```

1 # src/mraf_gs.py, line 164-213 (精简)
2 def _update_flat_wgs_weights(weights, farfield_amp, update_region,
3                               xp, backend, feedback_exponent,
4                               clip_min, clip_max, normalize_weights):
5     """WGS flat_local 权重更新."""
6     eps = np.float32(1e-12)
7
8     # 1. 提取 mask_flat 内的焦面振幅
9     amp_flat = farfield_amp[update_region]
10
11     # 2. 计算 flat 区平均振幅
12     amp_mean = xp.mean(amp_flat)
13
14     # 3. 核心公式: 每个像素的修正系数
15     # ratio = mean(|E_flat|) / |E_pixel|
16     # 如果某像素比平均暗 -> ratio > 1 -> 权重增大 -> 下次投影时要求更高
17     # 如果某像素比平均亮 -> ratio < 1 -> 权重减小 -> 下次投影时要求更低
18     ratio = amp_mean / xp.maximum(amp_flat, eps)

```

```

19     updated = weights[update_region] * xp.power(ratio, feedback_exponent)
20
21     # 4. Clip: 防止权重跑飞 (默认 0.5 ~ 1.5)
22     updated = xp.clip(updated, clip_min, clip_max)
23
24     # 5. 均值归一化: 保持所有权重的平均值为 1.0
25     if normalize_weights:
26         updated = updated / xp.mean(updated)
27
28     weights[update_region] = updated
29     return weights

```

7.1 直观理解

假设 mask_flat 内某个像素振幅是 mean 的 0.8 倍, exponent=0.8:

$$\text{ratio} = 1.0/0.8 = 1.25$$

$$\text{weight_new} = \text{weight_old} \times 1.25^{0.8} = \text{weight_old} \times 1.20$$

这个像素的权重增加了 20%。下次 MRAF 投影时, 该像素被要求匹配更高目标振幅 → 迭代推动它变亮 → 趋近平坦。

8 主迭代循环—完整流程

文件: src/mraf_gs.py, run_refinement() 函数

每轮迭代做 5 件事:

```

1  # src/mraf_gs.py, line 538-609 (精简, 去掉注释和日志)
2  for it in range(1, total_iters + 1):          # total_iters = 200
3
4      # (1) 前向传播: DOE -> 焦面
5      field = input_amp * exp(1j * phase)        # DOE 面复振幅
6      farfield = forward_fft(field)              # FFT -> 焦面复振幅
7      farfield_amp = abs(farfield)               # 焦面振幅
8
9      # (2) WGS 权重更新 (每 5 轮一次)
10     if it % update_every == 0:
11         weights = _update_flat_wgs_weights(     # 只在 mask_flat 内更新
12             weights, farfield_amp, update_region,
13             exponent=0.8, clip=[0.5, 1.5], normalize=True)
14
15     # (3) 加权目标振幅
16     target_weighted = base_target.copy()
17     target_weighted[mask_flat] *= weights       # flat 区 x WGS 权重

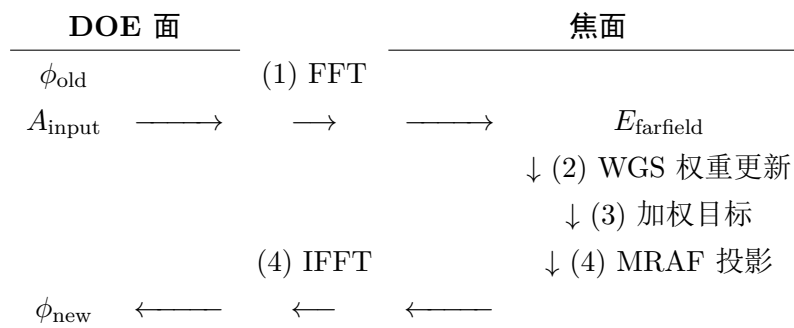
```

```

18
19 # (4) MRAF 投影 + 反传 + 相位提取
20 projected = _project_farfield(          # 三区投影
21     farfield, target_weighted,
22     mraf_factor=0.4, bg_factor=0.9)
23 nearfield = backward_fft(projected)    # IFFT -> DOE 面
24 phase     = angle(nearfield) % 2pi    # 提取新相位
25
26 # (5) 指标记录 (每 10 轮)
27 if it % metrics_interval == 0:
28     记录 RMS, size50, 效率...

```

数据流示意图:



9 ROM $\rightarrow$ SLM 一复振幅插值

文件: convert_to_slm.py

计算用的 2048×2048 相位不能直接送到 SLM (1024×1024)。需要裁切 + 缩放到 SLM 分辨率和像素间距。

```

1 # convert_to_slm.py, line 84-106 (精简)
2 # 1. 根据 SLM 物理尺寸 (17.4mm) / 计算网格像素间距 (20.8um) -> 裁切 837x837 像素
3 phys_mm = 1024 * 17.0e-3          # SLM 物理尺寸 = 17.4mm
4 crop_px = int(phys_mm / (dx_doe_um * 1e-3)) # 裁切像素数 = 837
5 phase_crop = phase[cy-418:cy+419, cx-418:cx+419] # 从 2048 中心裁出
6
7 # 2. * 关键: 复振幅插值 (不是直接插值相位!)
8 zoom_ratio = 1024 / 837          # ~ 1.22x 放大
9 cfield = np.exp(1j * phase_crop) # 转为复振幅
10 real_z = zoom(cfield.real, zoom_ratio, order=3) # 实部 cubic 插值
11 imag_z = zoom(cfield.imag, zoom_ratio, order=3) # 虚部 cubic 插值
12 phase_slm = np.arctan2(imag_z, real_z)          # 从插值后的复振幅恢复相位
13 phase_slm = phase_slm % (2*pi)

```

为什么不能直接插值包裹相位: 包裹相位在 $0 \leftrightarrow 2\pi$ 处有跳变, cubic spline 会把跳变当成高频信号产生巨大振铃。先转成连续平滑的 $\exp(i\phi)$ 分量, 插值完再恢复相位, 完全避免这个问题。

题。

10 闪耀光栅—让光斑偏离中心

在 DOE 相位上叠加线性斜坡，焦面光斑就会平移。

```

1 # 光栅相位:  $\phi_{\text{grating}}(x) = 2\pi \cdot (dx / \text{focal\_dx}) \cdot x / N$ 
2 focal_dx_um = 2.5      # 焦面像素间距
3 shift_x_um  = 400.0    # 想偏移 400um
4 shift_y_um  = 200.0    # 想偏移 200um
5
6 periods_x = shift_x_um / focal_dx_um  # = 160 个周期 (跨 2048 像素)
7 periods_y = shift_y_um / focal_dx_um  # = 80 个周期
8
9 x = np.arange(N) - N//2
10 X, Y = np.meshgrid(x, x)
11 grating = 2*pi * (periods_x * X + periods_y * Y) / N
12
13 phase_with_grating = (phase_doe + grating) % (2*pi)

```

物理原理: 线性相位斜坡 = 平面波倾斜入射 = 焦面光斑侧移。1 个光栅周期 (跨整个 DOE 孔径) = 焦面偏移 1 个像素 = $2.5\mu\text{m}$ 。

11 指标计算—怎么判断做得好不好

文件: src/metrics.py

```

1 # src/metrics.py, line 86-122 (精简)
2 def compute_metrics(intensity, masks, x_um, y_um, iteration):
3     I = intensity
4     mask_flat = masks["mask_flat"]
5
6     # (1) 归一化:  $I_{\text{norm}} = I / \text{mean}(I[\text{mask\_flat}])$ 
7     # 不以全局最大值为基准, 因为那会被偶然的 hot pixel 拉偏
8     I_n = I / np.mean(I[mask_flat])
9
10    # (2) RMS 不均匀度 (最核心指标)
11    flat_vals = I_n[mask_flat]
12    flat_rms = np.std(flat_vals) / np.mean(flat_vals)
13    # 例如: std=0.0087, mean=1.0 -> RMS = 0.87%
14
15    # (3) size50: 中心剖面半高全宽
16    profile_x = I_n[N//2, :]
17    size50_x = width_at_level(x_um, profile_x, level=0.5)
18    # 从中心向两侧找到 I=0.5 的 crossing 点, 线性插值

```

```

19
20 # (4) 效率: signal 区内能量 / 总能量
21 efficiency = np.sum(I[mask_signal]) / np.sum(I)
22
23 return { "flat_rms": flat_rms, "size50_x": size50_x, ... }

```

RMS 定义: $\text{std}(\text{mask_flat}) / \text{mean}(\text{mask_flat})$, 只算 flat core 内的像素。对于 beam=3.8mm, 200 轮后 RMS 0.71%。

12 参数速查表

参数	值	含义
λ	532 nm	激光波长
f	200 mm	透镜焦距
N	2048	计算网格
focal_dx	2.5 μm	焦面像素间距
beam (实测)	3.8 mm	$1/e^2$ 强度直径
目标 $W_{50} \times H_{50}$	$330 \times 120 \mu\text{m}$	平顶 50% 尺寸
δ_x / δ_y	15 / 8 μm	边缘过渡半宽
mraf_factor	0.4	free 区衰减系数
bg_factor	0.9	背景衰减 (接近 keep)
feedback_exponent	0.8	WGS 修正力度
weight_clip	[0.5, 1.5]	权重裁剪范围
update_every	5	每 N 轮更新一次权重
num_iters	200	总迭代轮数

13 文件索引

想了解什么	去看哪个文件
初始相位怎么来的	<code>make_phase0.py</code> (RD 公式)
目标光斑怎么定义	<code>src/rtad_target.py</code> (升余弦 + mask)
FFT 传播怎么写的	<code>src/propagation.py</code>
迭代循环逻辑	<code>src/mraf_gs.py</code> → <code>run_refinement()</code>
焦面投影公式	<code>src/mraf_gs.py</code> → <code>_project_farfield()</code>
WGS 权重怎么更新	<code>src/mraf_gs.py</code> → <code>_update_flat_wgs_weights()</code>
怎么转成 SLM 格式	<code>convert_to_slm.py</code>
光斑质量怎么量化	<code>src/metrics.py</code> → <code>compute_metrics()</code>
CLI 参数怎么传	<code>run_rtad_mraf_gs_case.py</code> → <code>parse_args()</code>
GPU/CPU 怎么选	<code>src/backend.py</code>
MATLAB 文件怎么读	<code>src/io_mat.py</code>